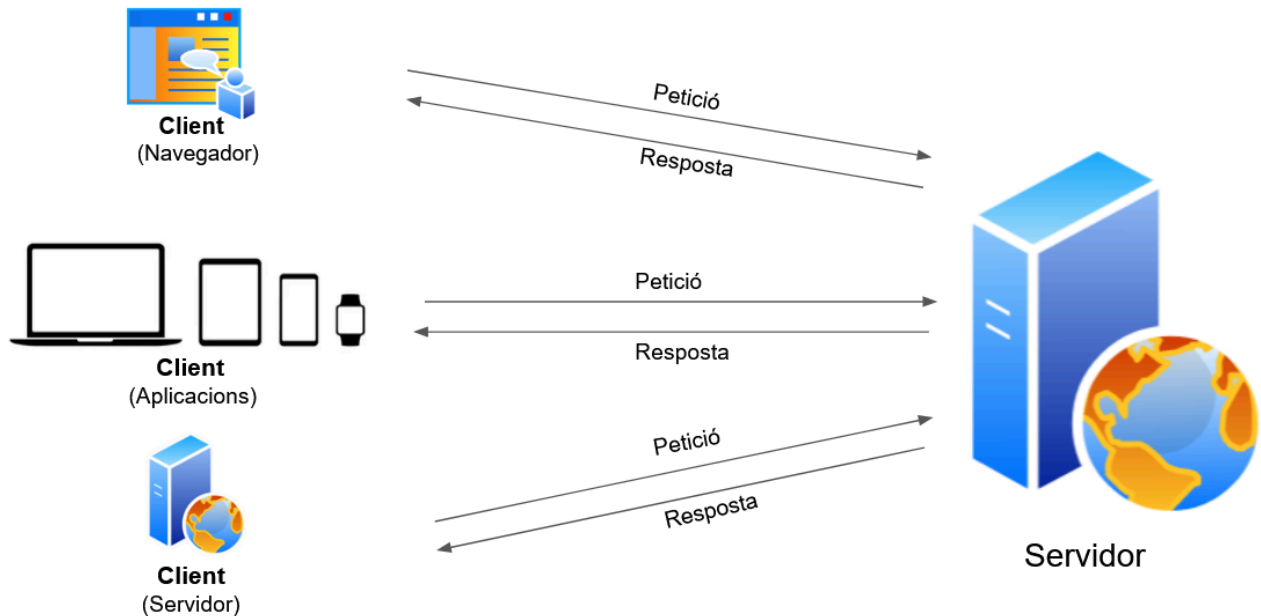


INTRODUCCIÓ

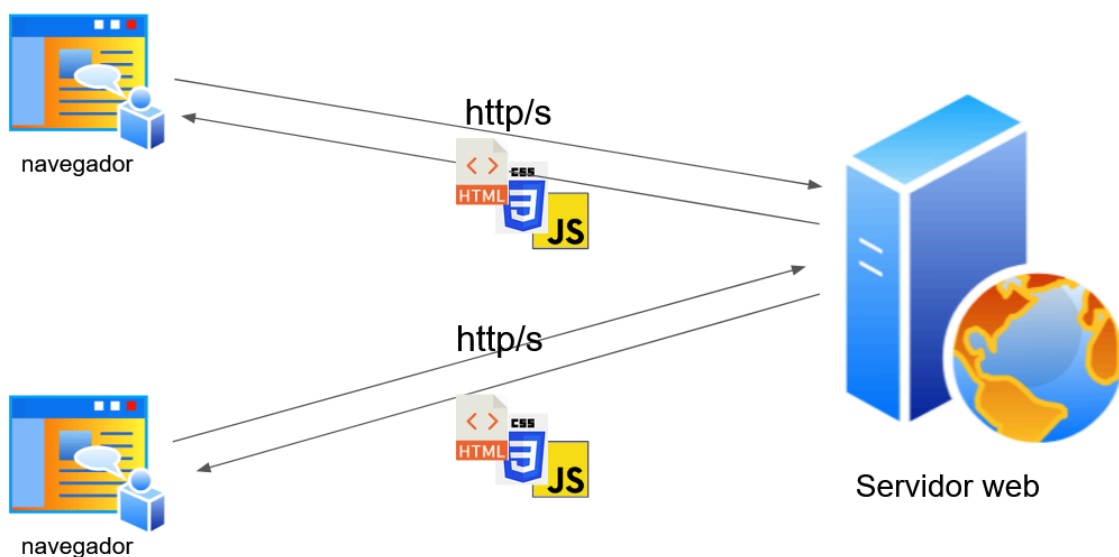
CLIENT / SERVIDOR

1. Arquitectura client / servidor



- **Client**: Qualsevol programa o dispositiu que **sol·licita un servei o recurs** a un servidor.
- **Servidor**: Programa o dispositiu que rep peticions, les processa i **proporciona un servei o recurs** als clients.

2. Arquitectura client / servidor en l'entorn web

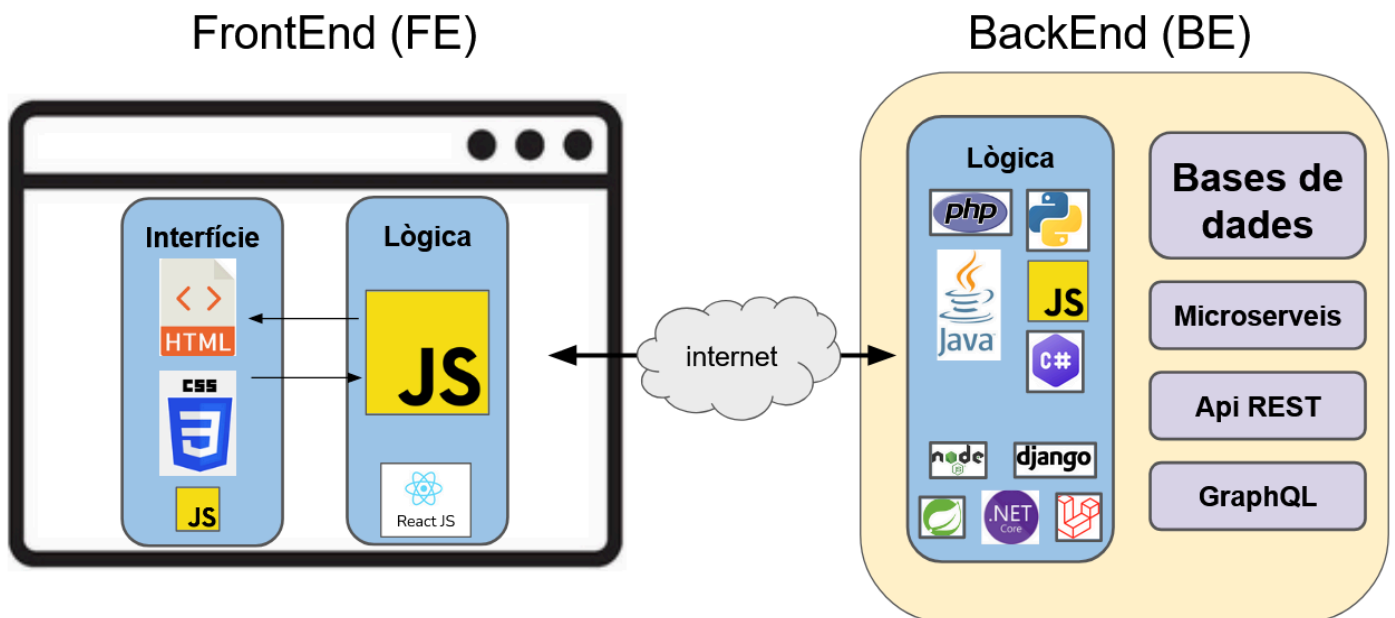


- **Client:** Tot aquella funcionalitat que està relacionat amb el **navegador web**:
 - Fer peticions de serveis o recursos al servidor web
 - El posterior tractament de la informació o recursos que es rep del servidor web.
- **Servidor:** Tot aquella funcionalitat que es troba allotjat a un **servidor web** remot que processa peticions del navegador i li retorna informació o recursos.

Les peticions entre **client** i **servidor** es fan a través del **protocol HTTP/S**

3. Frontend (FE) i Backend (BE)

Habitualment el desenvolupament de software es divideix en dos parts:



- **FrontEnd:**
 - Està relacionat amb **l'entorn client**.
 - Desenvolupament de la part visible i interactiva per a l'usuari (**UI/UX**)
 - Les tecnologies que s'utilitzen: **Html, css i javascript**.
 - El codi s'executa **al navegador** de l'usuari
 - **Insegur** (visible i modificable per l'usuari)
 - Exemple: Validar un formulari abans d'enviar-lo

- **BackEnd:**
 - Està relacionat amb **l'entorn servidor**.
 - Desenvolupament de la part que processa dades, lògica de negoci i bases de dades, seguretat,.....
 - Utilitza llenguatges com **Python, PHP, Java, C#, Node.js,...**
 - El codi **s'executa al servidor**
 - **Segur** (l'usuari no veu el codi)
 - Exemple: Processar un pagament o l'autenticació d'un usuari

4. Frontend

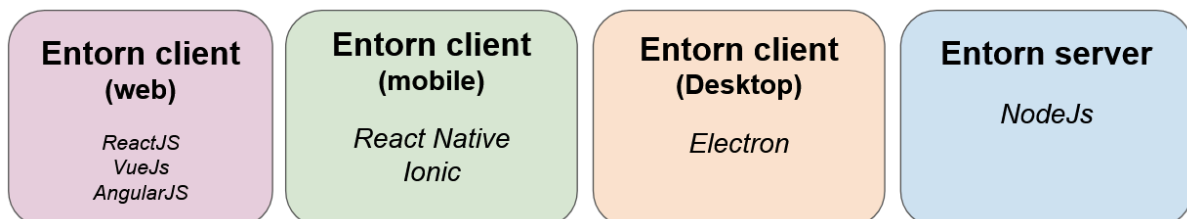
4.1 Javascript

És un llenguatge de programació que presenta les següents característiques:

- És **interpretat** (o compilat just-in-time)
- **No tipat** (no definim tipus a les variables)
- Orientats a objectes
- Amb funcions de primera classe (les funcions es poden guardar en variables, es poden passar per referència,....)
- Multi-paradigma: orientats a objectes, imperatius, funcionals, ...
- L'estàndard (la definició de les regles) per a JavaScript és ECMAScript (ECMA-262)

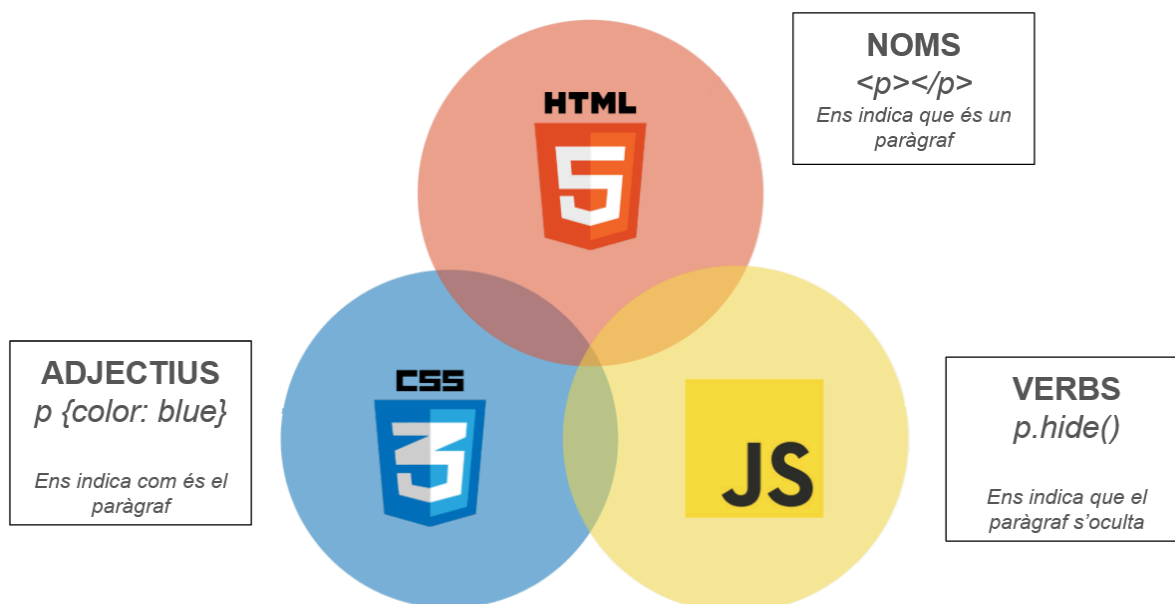
Javascript i java son dos llenguatges de programació diferents.

Ha deixat de ser un llenguatge utilitzat exclusivament en l'entorn del navegador web.



4.2 Javascript en el FE dins l'entorn web

És un llenguatge de programació d'alt nivell que permet l'interactivitat i implementar funcions complexes (variables, operacions, condicionals, bucles,...) en pàgines web a través del navegador.



4.2.1 - APIs

Es poden utilitzar moltes APIs (Application programming interface) disponibles per donar funcionalitat addicionals.

Les APIs no són part del llenguatge sinó que són implementacions que es poden utilitzar des de javascript.

- **del navegador:** estan integrats al navegador més moderns.
 - DOM: Per manipular l'html i el css de la pàgina web de forma dinàmica.
 - FETCH: Per obtenir dades del servidor
 - AUDIO/VIDEO: Per interactuar amb elements multimedia
 - GEOLOCALITACIÖ: Per obtenir la ubicació del lloc on estàs.
 - EMMAGATZEMATGE: Guardar dades en cookies, localStorage o sessionStorage
 - etc
- **de tercers:** no estan integrats al navegador i en general és necessari obtenir-los des d'una font.
 - Api de Twitter, google maps, facebook,...

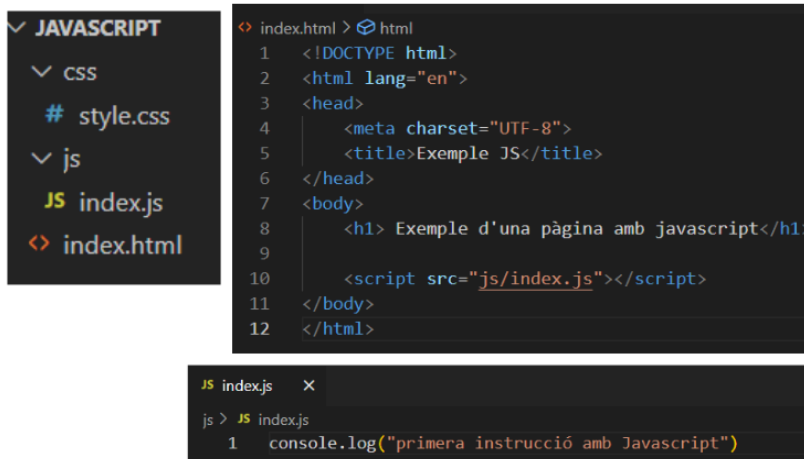
4.2.2 - Com utilitzem Javascript

- El javascript té moltes formes d'utilitzar-se en les pàgines web.
 - **Intern**
 - S'ha d'utilitzar una etiqueta `<script>` dins del body
 - Dins d'aquesta etiqueta podem fer servir javascript: definir variables, funcions, condicions, bucles,....
 - S'aconsella que l'etiqueta script estigui al final del document abans de tancar l'etiqueta body

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Exemple JS</title>
</head>
<body>
  <h1> Exemple d'una pàgina amb javascript</h1>

  <script>
    console.log("primera instrucció amb Javascript")
  </script>
</body>
</html>
```

- **External**
 - Es pot crear un fitxer extern amb el format `.js`
 - Podem cridar aquest fitxer a través de `<script src="nom_fitxer.js"></script>`
 - Es recomana igualment tenir aquesta etiqueta abans del tancament del *body*.
- Recordeu tenir una bona estructura del projecte amb carpetes pel css i el javascript.

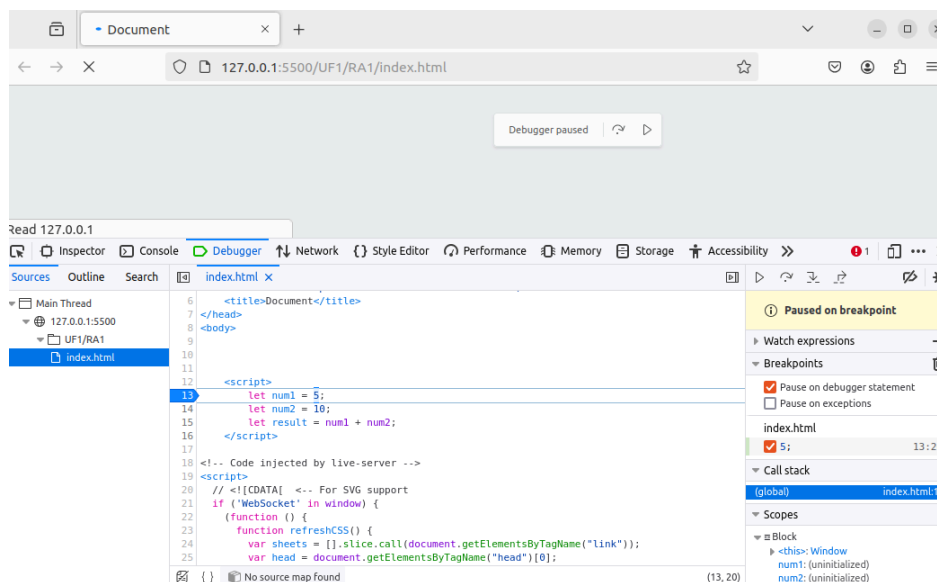


4.2.3 - Debugar javascript

- **Debugger del navegador**

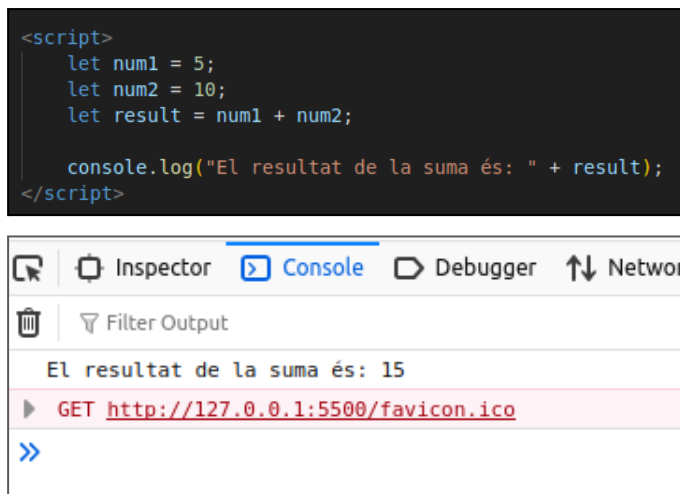
El navegador ens permet posar punts d'interrupció (breakpoints) per poder parar en un punt concret del codi en temps d'execució amb l'objectiu de poder detectar amb més facilitat els errors.

Pots comprovar valors de variables, resultat d'operacions matemàtiques, si entra en un condicional, etc ...

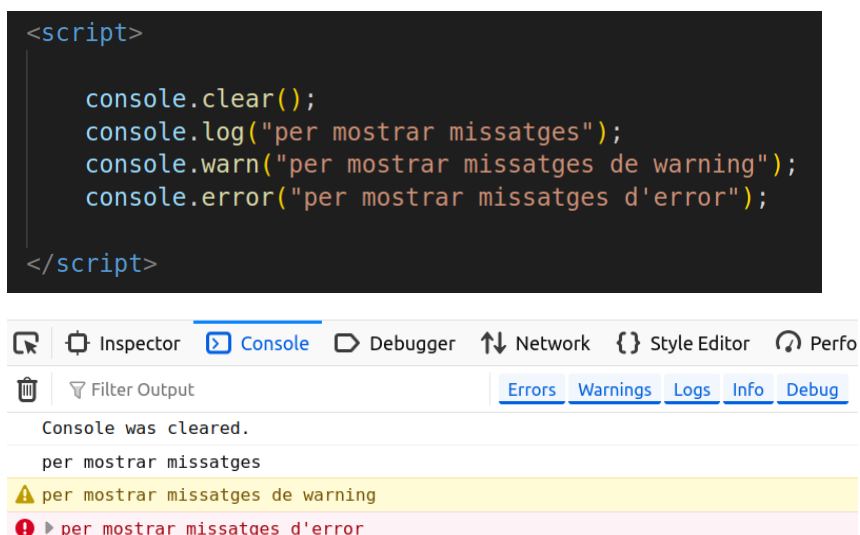


- **Console**

Per mostrar la informació a través de JS.



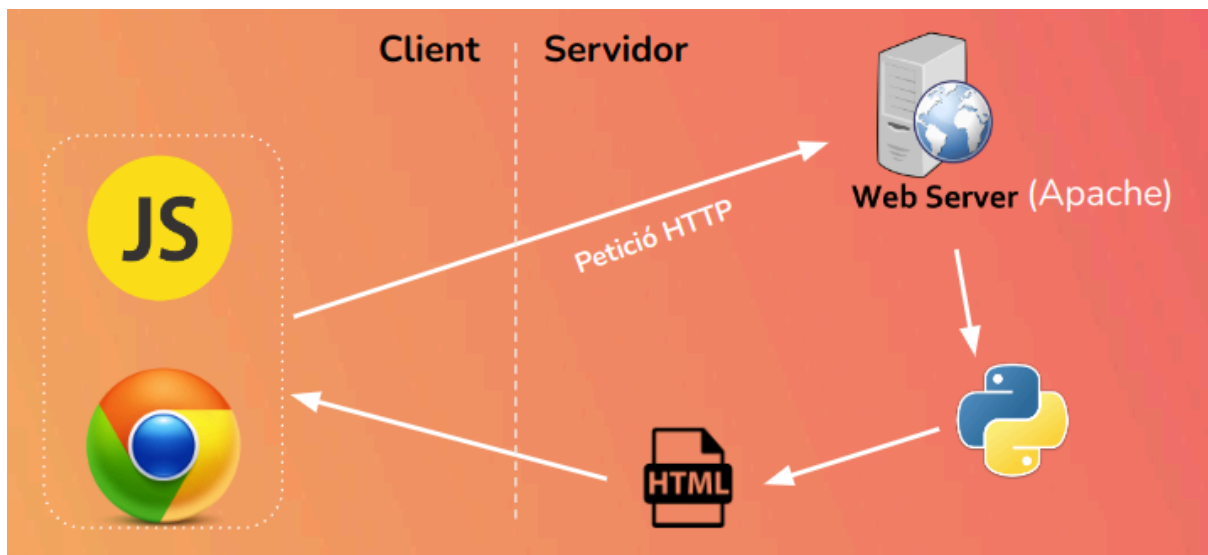
Tenim diferents opcions del console segons la importància del que volem rastrejar o mostrar.



5. Backend

5.1 Diferencia entre execució de còdi en l'entorn client i servidor

En un context web, un servidor web és el que executa còdi i retorna una resposta al client. Per exemple, PHP és una de les tecnologies que s'executen al costat del servidor.



Exemple:

Un script de Javascript que modifica el color de fons d'una botonera en el moment en que el ratolí clica el botó estaria codificat al costat del client i s'executaria en el navegador de l'usuari.

Un script de Python que genera contingut dinàmicament a partir d'una base de dades que s'executa en el servidor web. L'usuari clica a un botó i aquest genera una pàgina nova amb dades extretes de la Base de Dades.

5.2 Generació dinàmica de pàgines web

Personalització: La generació dinàmica de pàgines web permet personalitzar el contingut segons les sol·licituds/consultes/peticions de l'usuari.

Actualitzacions en temps real: Permet actualitzar la informació en temps real sense necessitat d'intervenció manual.

Diferències amb pàgines web estàtiques: Les pàgines web estàtiques tenen contingut fix que no es modifica a menys que es modifiqui manualment (programador modifiquen les línies de còdi). No és personalitzable segons usuari.

Pàgines web dinàmiques: El contingut es genera al moment de la sol·licitud de l'usuari, permetent personalitzar i actualitza la informació en temps real.

Exemple pàgina estàtica:

<https://uxenie.com/>

(probar a inspeccionar i en el footer, veure que el text de **Leandro** està a la mateixa pàgina)

5.3 Mecanismes d'execució de còdi en servidors web

Tecnologies:

Llenguatge de programació: PHP, Python, Ruby on Rails, etc. Aquests llenguatges permeten interactuar amb bases de dades, processar sol·licituds (queries) i generar respostes dinàmiques.

Servidors d'aplicacions: Apache Tomcat, IIS, etc. Aquests servidors permeten executar aplicacions web complexes que requereixen més funcionalitats que un servidor web bàsic.

5.4 Funcionalitats dels servidors d'aplicacions

Funcionalitats:

Gestió de transaccions: Els servidors d'aplicacions gestionen transaccions i sessions, essencial per les aplicacions web complexes com botigues en línia.

Integració amb servidors web: S'integren amb servidors web per proporcionar serveis com autenticació i gestió de sessions.

5.5 Llenguatges i tecnologies en entorn servidor

Llenguatges de programació:

PHP: Un llenguatge de programació del costat del servidor molt popular per generar contingut dinàmic.

Python: Altre llenguatge de programació del costat del servidor àmpliament utilitzat, especialment amb frameworks com Django.

Ruby on Rails: Un framework de Ruby para desarrollar aplicaciones web del lado del servidor.

5.6 Integració de llenguatges de marques amb llenguatges de programació

Mecanismes d'integració:

Plantilles: Els llenguatges de marques (HTML, CSS, XML, etc.) s'integren amb els llenguatges de programació a través de plantilles que inclouen còdi del costat del servidor. Per exemple, un servidor web que utiliza PHP per generar contingut dinàmic i després ho integra en una plantilla HTML.

5.7 Eines de programació en entorn servidor

Eines:

Apache Tomcat: Un servidor d'aplicacions que permeten executar aplicacions web escrites en Java.

IIS: Un servidor d'aplicacions de Microsoft que soporta varies tecnologies, incluent ASP.NET.

Nginx: Un servidor web i proxy invers que també pot actuar como servidor d'aplicacions.

5.8 Exemple pràctic de client-servidor

Exemple:

Pàgina web de comerç electrònic: El client (navegador de l'usuari) sol·licita informació i realitza comandes, mentre el servidor processa les transaccions i gestiona les sessions de l'usuari.

5.9 Avantatges i desavantatges del model cliente-servidor

Avantatges:

- Personalització: Permet personalitzar el contingut per a cada usuari.
- Escalabilitat: Facilita l'escalabilitat al permetre la distribució de la càrrega entre varis servidors.

Desavantatges:

- Dependència del servidor: El client depèn del servidor per a obtenir els recursos i serveis.
- Seguretat: Pot haver riscos de seguretat si no s'implementen mesures adequades, com l'autenticació i el xifrat.

5.10 Proves d'integració en client-servidor

Tipus de proves:

Proves unitaries: Verificar que cada component del sistema funcioni correctament.

Proves d'integració: Verificar que els components del sistema es comuniquin correctament entre sí.

Proves de rendiment: Verificar que el sistema pugui gestionar la càrrega esperada.

EXERCICIS

Respondre les preguntes segons la teoria exposada entre el document i les diapositives. Explicar-ho amb paraules pròpies sense copiar literalment el text extret (dels apunts o d'internet).

1. Explica la diferència entre l'execució del còdi en el costat del cliente i en el costat del servidor. Proporciona un exemple en cada cas.
2. Descriu els beneficis de la generació dinàmica de pàgines web en comparació amb la inclusió de sentències de guions en les pàgines web estàtiques. Proporciona un exemple en cada cas.
3. Identifica i descriu els mecanismes d'execució de còdi en els servidors web. Proporciona exemples de tecnologies involucrades.
4. Descriu les funcionalitats que aporten els servidors d'aplicacions i la seva integració amb els servidors web. Proporciona un exemple bàsic.
5. Verifica els mecanismes d'integració dels llenguatges de marques (HTML, CSS, XML, etc) amb els llenguatges de programació en entorn servidor. Proporciona un exemple de com s'aconseguiria aquesta integració.